

P1272R0

Byteswapping for fun&&nuf

Published Proposal, 2018-07-12

This version:

<https://wg21.link/P1272R0>

Author:

[Isabella Muerte](#) (Target)

Audience:

SG6, SG14, LEWG

Toggle Diffs:

☐ Hide deleted text

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1272R0](#)

Current Source:

slurps-mad-rips/papers/proposals/byteswap.bs

Implementation:

slurps-mad-rips/byteswap

Table of Contents

1 Revision History

1.1 Revision 0

2 Motivation

3 Design Considerations

3.1 Synopsis

4 Wording

4.1 Feature Testing

5 Acknowledgement

References

Informative References

§ 1. Revision History

§ 1.1. Revision 0

Initial release. 🎉

§ 2. Motivation

Proposal [\[P0553r2\]](#) gives several bit operations to perform on integer types, such as popcount, or bit rotation. Despite these new operations provided to C++ developers, we still cannot swap (i.e., reverse) the bytes of builtin integer types in a performant way (i.e., one instruction or less) without resorting to compiler intrinsics. Currently, most CPU architectures provide single instructions for a byte swap. For those that don't, falling back on existing operations is more than amenable. We should, however, endeavor to standardize existing practice.

Note: The phrase one instruction or less refers to compilers inserting at most one instruction, and at the very least removing any instructions due to optimizations.

§ 3. Design Considerations

The design for the byteswap free function is quite simple. It takes any integer type and swaps its byteorder to the reverse of its current state. Additionally, it *only* takes integer types, requiring users to openly `bit_cast` their non-integers to integers in blatant heresy to the Worm-Seethe. How utterly disgraceful.

Note: It is intended that the byteswap function, despite swapping bytes, be placed into the `<bit>` header.

§ 3.1. Synopsis

The function's full specification is:

```
namespace std {  
    template <class IntegerType>  
        constexpr IntegerType byteswap (IntegerType value) noexcept;  
}
```

Where `std::is_integral_v<IntegerType>` is `true`.

§ 4. Wording

The following is wording for the library section.

```
namespace std {  
    template <class IntegerType>  
        constexpr IntegerType byteswap (IntegerType value) noexcept;  
}
```

1. Constraints This function shall not participate in overload resolution unless:

- `std::is_integral_v<IntegerType>` is `true`

2. Returns

An object of type `IntegerType` whose representation is in the reverse order of the object representation of `value`.

§ 4.1. Feature Testing

The `__cpp_lib_byteswap` feature test macro should be added.

§ 5. Acknowledgement

Thanks to Nicole Muzzuca for feedback on writing this proposal.

§ References

§ Informative References

[P0553r2]

Jens Maurer. [Bit operations](https://wg21.link/p0553r2). 23 November 2017. URL: <https://wg21.link/p0553r2>